

**Departamento de Engenharia Elétrica e de Computação EESC USP SEL0337 –
Projetos em Sistemas Embarcados**

Prática #3 – GPIO, Python e Computação Paralela

Aluno: Pedro Tadeu Barbosa Guimarães Pereira – 12683912

1. Introdução

A prática teve como objetivo desenvolver aplicações em Python utilizando os pinos GPIO da Raspberry Pi, explorando leitura de entradas digitais, acionamento de saídas, tratamento de eventos, PWM, sensores e conceitos de computação concorrente por meio de threads.

Ao longo dos três checkpoints foram desenvolvidas aplicações progressivamente mais complexas. Inicialmente foram realizados o acionamento de um LED por botão e uma contagem regressiva. Em seguida, foi implementado o controle de brilho de um LED utilizando PWM e uma aplicação com sensor ultrassônico HC-SR04 e quatro LEDs. Por fim, a aplicação do sensor ultrassônico foi adaptada para utilizar múltiplas threads, permitindo a execução concorrente da leitura da distância, controle dos LEDs e apresentação dos dados no terminal.

2. Checkpoint 1 – GPIO e programação em Python

No primeiro checkpoint foram aplicados conceitos básicos de programação em Python associados ao controle das GPIOs da Raspberry Pi. A primeira aplicação consistiu no acionamento de um LED através de um botão. O botão foi configurado como entrada utilizando o resistor Pull-Up interno da Raspberry Pi. Dessa forma, ao pressionar o botão, o programa detectava a mudança de estado e acionava o LED, desligando-o quando o botão era solto.

Foi utilizada a biblioteca RPi.GPIO, incluindo detecção de eventos e tratamento da interrupção por CTRL+C. Ao final da execução, a função GPIO.cleanup() foi utilizada para liberar corretamente os pinos GPIO. A segunda aplicação consistiu em uma contagem regressiva. O usuário informava no terminal um tempo em segundos e o programa realizava a contagem no formato MM:SS. Foram utilizadas estruturas de tratamento de erros para evitar o encerramento do programa caso fossem inseridos valores inválidos. Ao final da contagem, o LED era acionado e permanecia aceso.

3. Checkpoint 2 – PWM e sensor ultrassônico

3.1 Controle de LED utilizando PWM

Inicialmente foi implementado um sinal PWM para controlar o brilho de um LED. A frequência utilizada foi de aproximadamente 1 kHz, enquanto o duty cycle podia ser informado pelo usuário através do terminal.

O duty cycle representa a porcentagem de cada período durante a qual o sinal permanece em nível lógico alto. Dessa forma, valores menores produzem menor brilho aparente no LED, enquanto valores maiores provocam aumento do brilho.

Para verificar experimentalmente o comportamento do sinal, a saída PWM foi observada utilizando um osciloscópio. A frequência permaneceu aproximadamente constante em 1 kHz, enquanto a relação entre os tempos em nível alto e baixo foi alterada conforme o duty cycle escolhido.

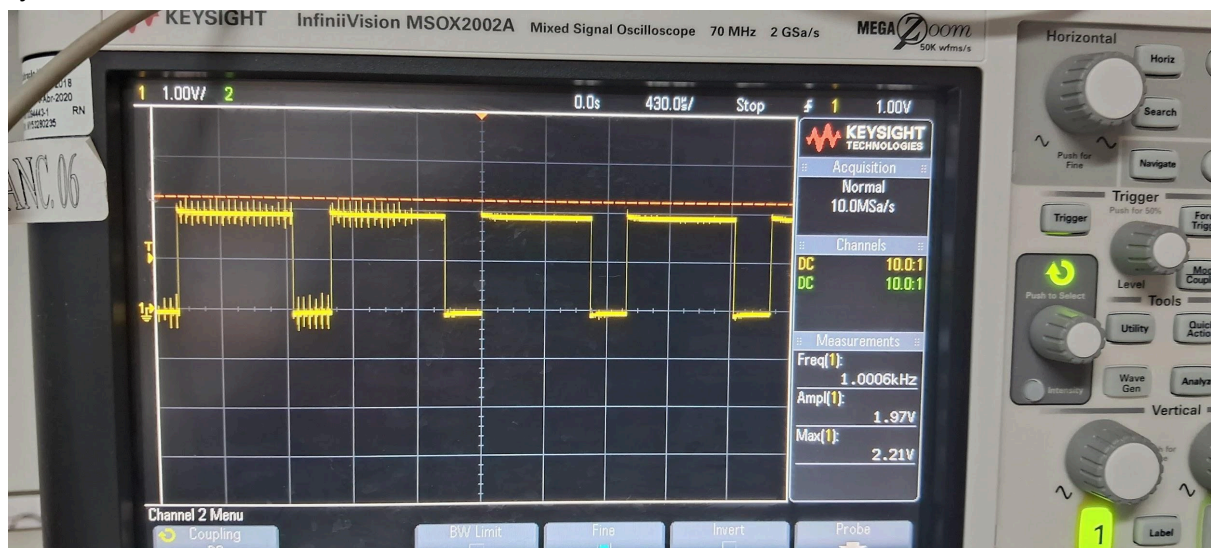


Figura 1 – Sinal PWM observado no osciloscópio com duty cycle elevado e frequência de aproximadamente 1 kHz.

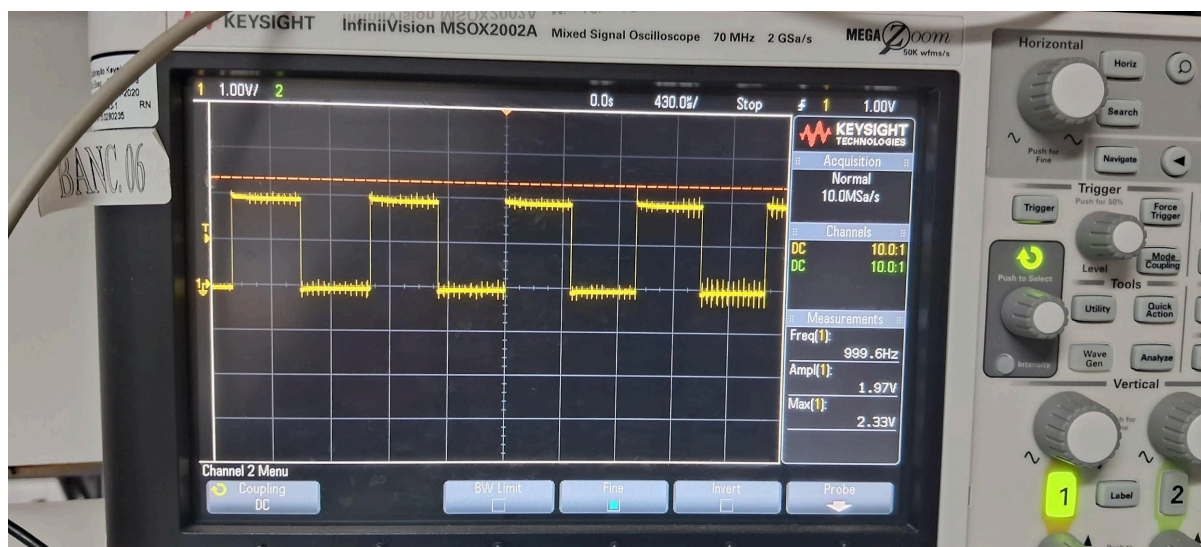


Figura 2 – Sinal PWM observado com duty cycle próximo de 50% e frequência de aproximadamente 1 kHz.

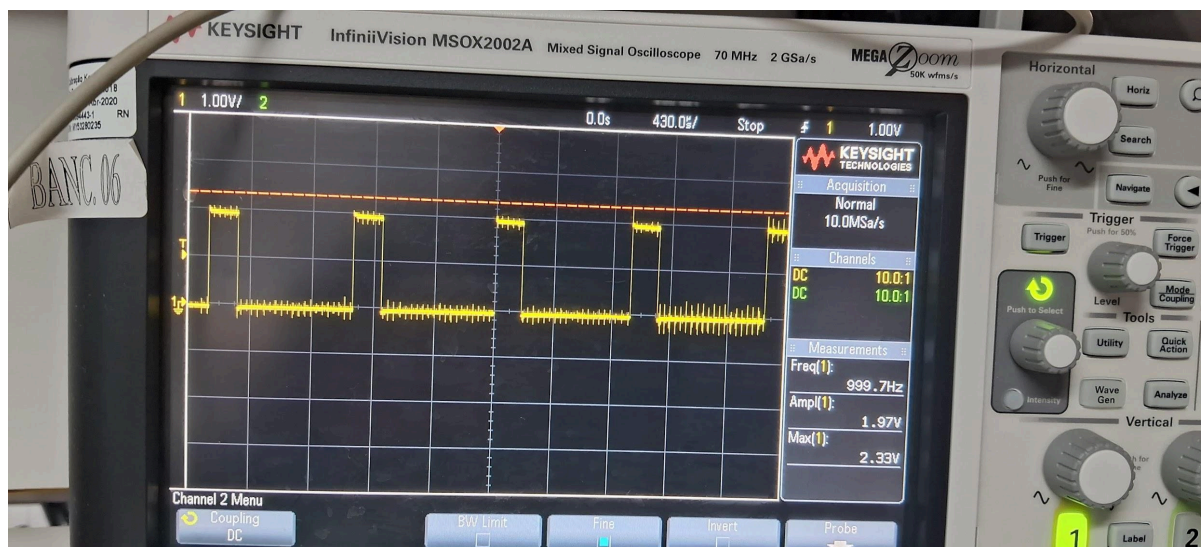


Figura 3 – Sinal PWM observado no osciloscópio com duty cycle reduzido e frequência de aproximadamente 1 kHz.

3.2 Sensor ultrassônico HC-SR04 e indicação por LEDs

A segunda aplicação do Checkpoint 2 utilizou o sensor ultrassônico HC-SR04 para medir a distância entre o sensor e um objeto. Para facilitar a implementação, foi utilizada a classe DistanceSensor da biblioteca gpiozero.

Foram conectados quatro LEDs às GPIOs 23, 24, 25 e 22 da Raspberry Pi. O sistema foi programado para aumentar o número de LEDs acesos conforme o objeto se aproximava do sensor.

A lógica implementada foi:

Distância medida	LEDs acesos
Maior que 40 cm	0
Entre 30 e 40 cm	1
Entre 20 e 30 cm	2
Entre 10 e 20 cm	3
Até 10 cm	4

A distância fornecida pela biblioteca `gpiozero` foi convertida de metros para centímetros antes da comparação. Além do acionamento dos LEDs, o valor medido era apresentado continuamente no terminal.

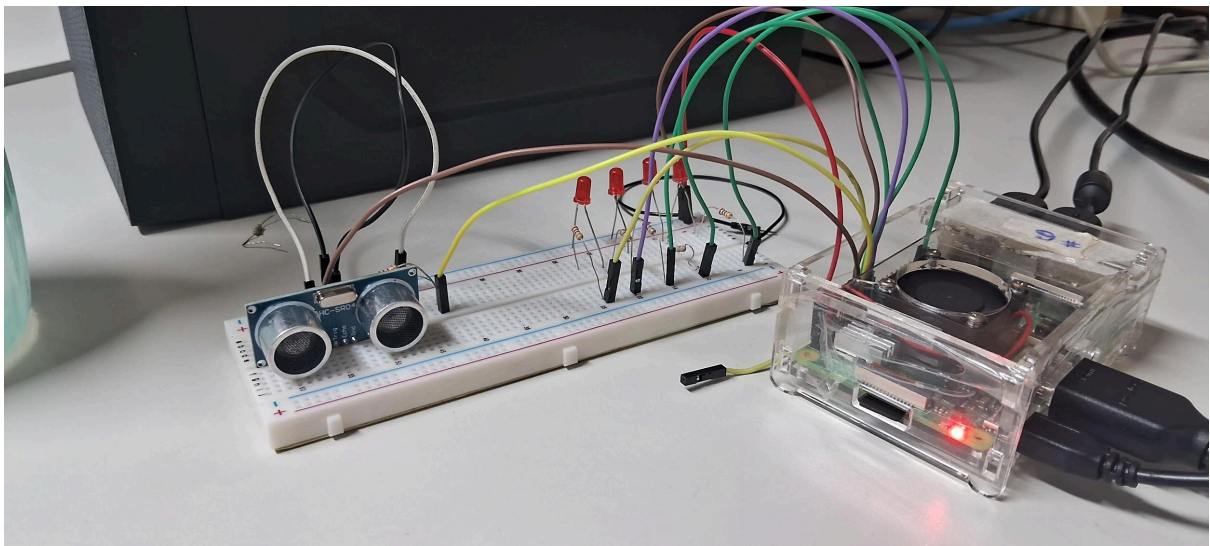


Figura 4 – Montagem da aplicação utilizando Raspberry Pi, sensor ultrassônico HC-SR04 e quatro LEDs indicadores.

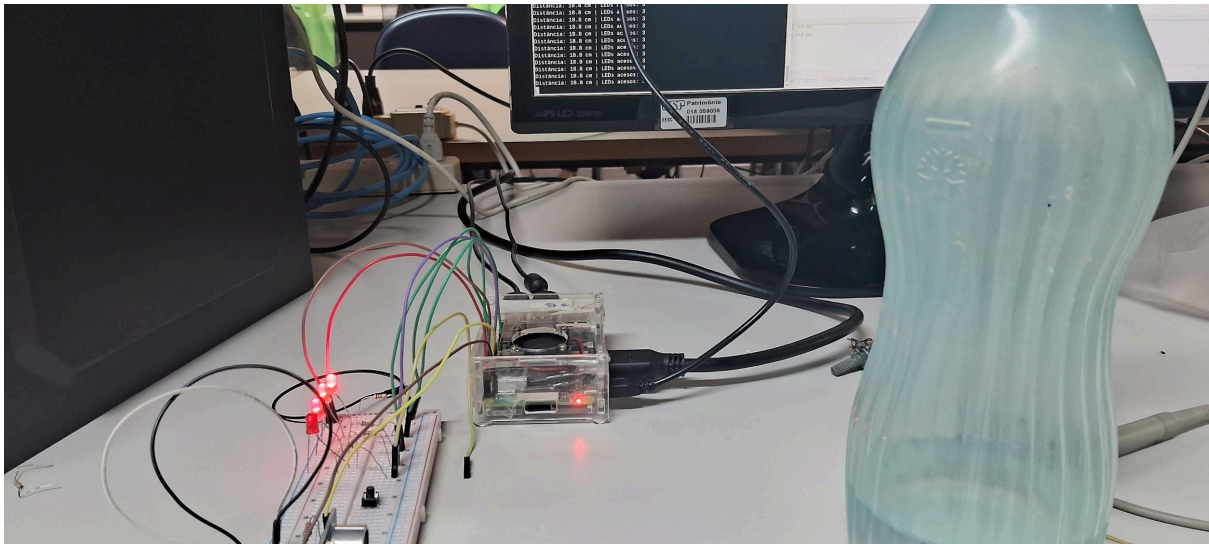


Figura 5 – Execução da aplicação com objeto a aproximadamente 18 cm do sensor, resultando no acionamento de três LEDs.

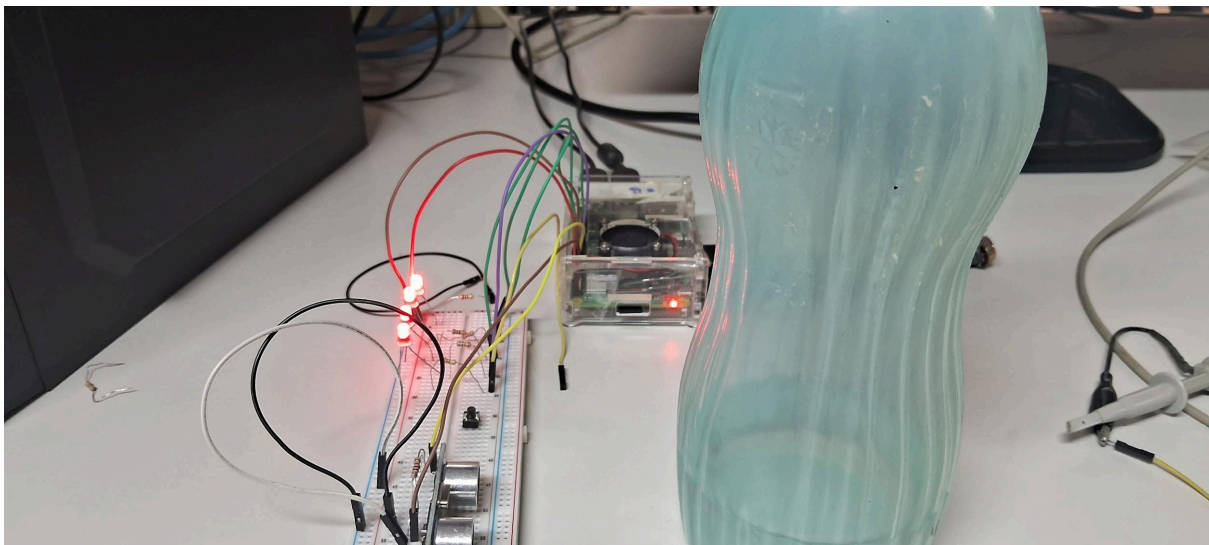


Figura 6 – Resposta visual da aplicação à aproximação de um objeto do sensor ultrassônico.

4. Checkpoint 3 – Aplicação utilizando threads

No terceiro checkpoint, a aplicação desenvolvida anteriormente com o sensor ultrassônico e os quatro LEDs foi reutilizada e modificada para utilizar multithreading. A aplicação foi dividida em três tarefas concorrentes. Uma thread ficou responsável pela leitura periódica do sensor ultrassônico, outra pelo controle dos LEDs em função da distância e uma terceira pela apresentação da distância medida no terminal.

A variável contendo a distância foi compartilhada entre as threads. Para controlar o acesso a esse recurso compartilhado, foi utilizado um mutex, implementado através de `threading.Lock()`. Dessa maneira, o acesso à variável ocorre de forma controlada. Também foi utilizado `threading.Event()` para sinalizar o encerramento das tarefas. Quando ocorre a interrupção por CTRL+C, o evento é ativado e as threads terminam seus respectivos laços. Em seguida, `join()` é utilizado para aguardar o encerramento das threads antes da liberação

dos recursos. O roteiro permite reutilizar as aplicações dos checkpoints anteriores como base para a aplicação com processos e/ou thread

A montagem física do Checkpoint 2 foi mantida no Checkpoint 3. Nesta etapa, as principais alterações foram realizadas no software, com a divisão das tarefas de leitura do sensor, acionamento dos LEDs e apresentação dos dados em threads independentes.

5. Multithreading e Multiprocessing

No multithreading, várias threads são executadas dentro de um mesmo processo e compartilham o mesmo espaço de memória. Isso facilita a troca de informações entre as tarefas, porém pode exigir mecanismos de sincronização, como mutexes, quando diferentes threads acessam um mesmo recurso. No multiprocessing, por outro lado, cada processo possui seu próprio espaço de memória. Essa abordagem proporciona maior isolamento entre as tarefas e pode permitir melhor utilização de diferentes núcleos da CPU em aplicações que possuem maior carga de processamento.

Para a aplicação desenvolvida neste trabalho foi escolhido o multithreading, pois as tarefas executadas envolvem principalmente operações de entrada e saída, como leitura do sensor, controle das GPIOs e impressão das informações no terminal. Além disso, as threads permitem compartilhar diretamente a variável que armazena a distância medida. O próprio roteiro destaca que threads podem ser uma solução adequada para GPIO, temporizações e operações de entrada/saída, enquanto processos oferecem maior isolamento e são úteis quando existe interesse em explorar múltiplos núcleos para determinadas cargas de processamento.

6. Conclusão

A prática permitiu aplicar diferentes conceitos relacionados ao desenvolvimento de sistemas embarcados utilizando Python e Raspberry Pi. Inicialmente foram exploradas operações básicas de entrada e saída digital, incluindo leitura de botão, acionamento de LED, tratamento de eventos e temporização. Posteriormente, foi implementado o controle de uma saída por PWM, cujo comportamento foi verificado experimentalmente utilizando um osciloscópio.

A utilização do sensor ultrassônico HC-SR04 juntamente com quatro LEDs permitiu desenvolver uma aplicação em que uma grandeza física medida pelo sistema é convertida em uma indicação visual da distância. Por fim, a aplicação foi adaptada utilizando multithreading, permitindo que a leitura do sensor, o controle dos LEDs e a apresentação dos dados fossem executados concorrentemente. Dessa forma, foi possível aplicar na prática conceitos de threads, compartilhamento de dados e sincronização por mutex.